Case Study 99

# BlockShield Security Pvt. Ltd

Blockchain-Based Data Security Platform

Ritesh Jadhav
150096724138
Jensen Huang
GitHub Link - Link
Google Sheet Link - Blockchain-Based Data Security Platform

## Section A : Business and Financial Analysis:

1. Calculate the total annual subscription revenue if 200 clients
2. Calculate the Net Benefit using: Net Benefit = Total Revenue – Annual Operational Cost – Initial Investment.

Case Study 99 : Blockchain-Based Data Security Platform at BlockShield Security Pvt. LTD

Students must analyze both financial feasibility and technical viability to evaluate whether launching the blockchain-based data security platform is profitable and sustainable.

**A. One-Time (Development & Launch) Costs**

| Cost Component | Amount (₹) |
|---|---|
| Blockchain development, | ₹10,000,000.00 |
| Cloud infrastructure, | ₹25,000,000.00 |
| Encryption systems, | ₹10,000,000.00 |
| Skilled cybersecurity professionals. | ₹15,000,000.00 |
| **Total Initial Cost** | ₹60,000,000.00 |

**B. Annual Operating Costs**

| Cost Component | Annual Cost (₹) |
|---|---|
| App maintenance & feature updates | ₹2,500,000.00 |
| Customer support & dispute resolution | ₹4,500,000.00 |
| Delivery partner support systems | ₹3,000,000.00 |
| Cybersecurity & data protection | ₹10,000,000.00 |
| **Total Annual Cost** | ₹20,000,000.00 |

**C. Annual Benefit**

| | |
|---|---|
| Number of Clients | 200.00 |
| Annual Subscription Fees | ₹300,000.00 |
| Annusl Reveue From Clients | ₹60,000,000.00 |

**D. Net Benefit for 2 Years** Net Benefit = Total Revenue – Annual Operational Cost – Initial Investment.

| Years | Profit |
|---|---|
| Net Benefit Year - 1 | -₹20,000,000.00 |
| Net Benefit Year - 2 | ₹40,000,000.00 |

Excel Sheet Link - Blockchain-Based Data Security Platform

## Section B : Technical and Data Analysis:

3. What security and blockchain variables (such as encryption success rate, system uptime, unauthorized access attempts, and transaction integrity) should be collected to evaluate platform effectiveness? Explain their importance.

| Sr No. | Variables | Category | Target | Importance and Business Impact |
|---|---|---|---|---|
| 1 | Encryption Success Rate | Security | >=99.97% | Measures the % of data blocks encrypted without error. Any failure means data may be stored in plain text, exposing it to breaches. Tracks health of the AES-256/RSA engine. Failure triggers SLA penalties and regulatory fines. |
| 2 | System Uptime | Reliability | >=99.99% | Tracks platform availability 24x7. Banks and hospitals cannot tolerate downtime. Even 1 hour outage at a bank can halt thousands of transactions. SLA commitments require 99.9%+ uptime. |
| 3 | Unauthorized access attempts | Threat Intel | >50/hr = alert | Counts rejected login/API/node attempts. Monitoring frequency and IP source patterns enables proactive threat detection and automatic IP blocking before a breach can escalate. |
| 4 | Transaction Process Speed | Performance | >=1000TPS | Measures data write/read throughput. Banks process millions of daily transactions. Low TPS causes bottlenecks and client SLA breaches. Enables capacity planning and node scaling decisions. |
| 5 | Transaction Integrity Score | BlockChain | 100% | Verifies SHA-256 hash consistency from origin to storage. Any hash |

| | | | | |
|---|---|---|---|---|
| | | | | mismatch = tampering. This is BlockShield's core tamper-proof value proposition. Even one failure can destroy client trust. |
| 6 | Incident Response Time | Operations | <4-5 minutes | Time from alert detection to response deployment (IP block, node quarantine, client notification). Faster response reduces breach blast radius and |
| 7 | Client Satisfaction Score | CRM | >=8.5/10 | Collected via surveys or AI sentiment analysis. Links technical KPIs to business outcomes. Scores below 7.0 are early churn indicators, enabling proactive customer success intervention. |

## 4. How can synthetic blockchain security and transaction data be generated using Python?

a. Since BlockShield has not yet launched, synthetic data is generated to test dashboards, train AI anomaly-detection models, and validate business reports.

| DataSet | Variables Covered | Rows Generated | Output File |
|---|---|---|---|
| Dataset 1 – Transactions | V1: Encryption Rate,<br>V4: TPS,<br>V5: Integrity | 1,000 rows | blockshield_Transaction.csv |
| DataSet 2 – Monitoring | V2: Uptime,<br>V3: Unauth Attempts,<br>V6: Response Time | 365 rows (1 per day) | blockshield_monitoring.csv |
| DataSet 3 – CRM | V7: Client Satisfaction Score, CLV, Renewal | 200 rows (1 per client) | blockshield_crm.csv |

b. Libraries Used in the Code and Why.

| Library | Function | Why Use thiS Library |
|---|---|---|

| | | |
|---|---|---|
| Pandas | Create & save DataFrames | Data handling + CSV read/write |
| Numpy | Statistical distributions | Normal, Poisson, Log-Normal |
| Hashlib | SHA-256 hashing | Real blockchain fingerprints |
| uuid | Unique transaction IDs | No duplicate tx_ids |
| random | Weighted random choices | Probabilities + seed(42) |
| datetime | Realistic timestamps | Full year of timestamps |
| matplotlib | Charts & graphs | Histograms + line charts |

## c. Python Source Code For CSV Generation

```
import pandas as pd
import numpy  as np
import hashlib, uuid, random
from datetime import datetime, timedelta

np.random.seed(42)
random.seed(42)

NUM_RECORDS = 1000
NUM_CLIENTS = 200
INDUSTRIES  = ['Banking', 'Healthcare', 'Government', 'IT']
START_DATE  = datetime(2024, 1, 1)

def generate_hash(data: str) -> str:
    return hashlib.sha256(data.encode()).hexdigest()
def random_timestamp(start: datetime, days: int = 365) -> datetime:
    return start + timedelta(
        days=random.randint(0, days),
        hours=random.randint(0, 23),
        minutes=random.randint(0, 59)
    )
```

---

```
print("Generating Transaction Records dataSet")
records = []

for _ in range(NUM_RECORDS):
```

```
    client_id = f'CLIENT_{random.randint(1, NUM_CLIENTS):03d}'
    industry  = random.choice(INDUSTRIES)
    timestamp = random_timestamp(START_DATE)
    data_size = round(np.random.lognormal(mean=5, sigma=1.2), 2)
    algo      = random.choice(['AES-256', 'RSA-2048', 'ChaCha20'])
    enc_ok    = np.random.choice([True, False], p=[0.9997, 0.0003])
    tps       = round(max(100, np.random.normal(loc=1200, scale=150)), 1)
    tampered  = np.random.choice([True, False], p=[0.005, 0.995])
    integrity = 0 if tampered else 1
    block_hash = generate_hash(f'{client_id}{timestamp}{data_size}')
    prev_hash  = generate_hash(f'{client_id}{timestamp}')


    records.append({
        'tx_id'               : str(uuid.uuid4()),
        'client_id'           : client_id,
        'industry'            : industry,
        'timestamp'           : timestamp.strftime('%Y-%m-%d %H:%M'),
        'data_size_kb'        : data_size,
        'encryption_algorithm' : algo,
        'encryption_success'   : enc_ok,
        'tps'                 : tps,
        'tx_integrity'        : integrity,
        'block_hash'          : block_hash,
        'prev_hash'           : prev_hash,
        'tamper_detected'     : tampered
    })

df_tx = pd.DataFrame(records)
print(f"   Done — {len(df_tx):,} records created.")
```

---

```
print("Generating System Monitoring Dataset")
monitoring = []


for i in range(365):
    date = (START_DATE + timedelta(days=i)).strftime('%Y-%m-%d')
    uptime = round(
        max(99.0, min(100.0, 100 - abs(np.random.normal(loc=0, scale=0.08)))), 4
    )
    unauth = max(0, int(np.random.poisson(lam=22)))
    alerts = max(0, int(np.random.poisson(lam=6)))
    response_min = round(abs(np.random.normal(loc=3.2, scale=1.0)), 2)
    nodes  = random.randint(48, 50)
```

```
    status = 'ALERT' if (uptime < 99.5 or unauth > 50) else 'Normal'
    monitoring.append({
        'date'                  : date,
        'uptime_pct'            : uptime,
        'unauthorized_attempts'  : unauth,
        'security_alerts'        : alerts,
        'incident_response_min'  : response_min,
        'active_nodes'           : nodes,
        'status'                 : status
    })

df_system = pd.DataFrame(monitoring)
print(f"   Done — {len(df_system):,} daily records created.")
```

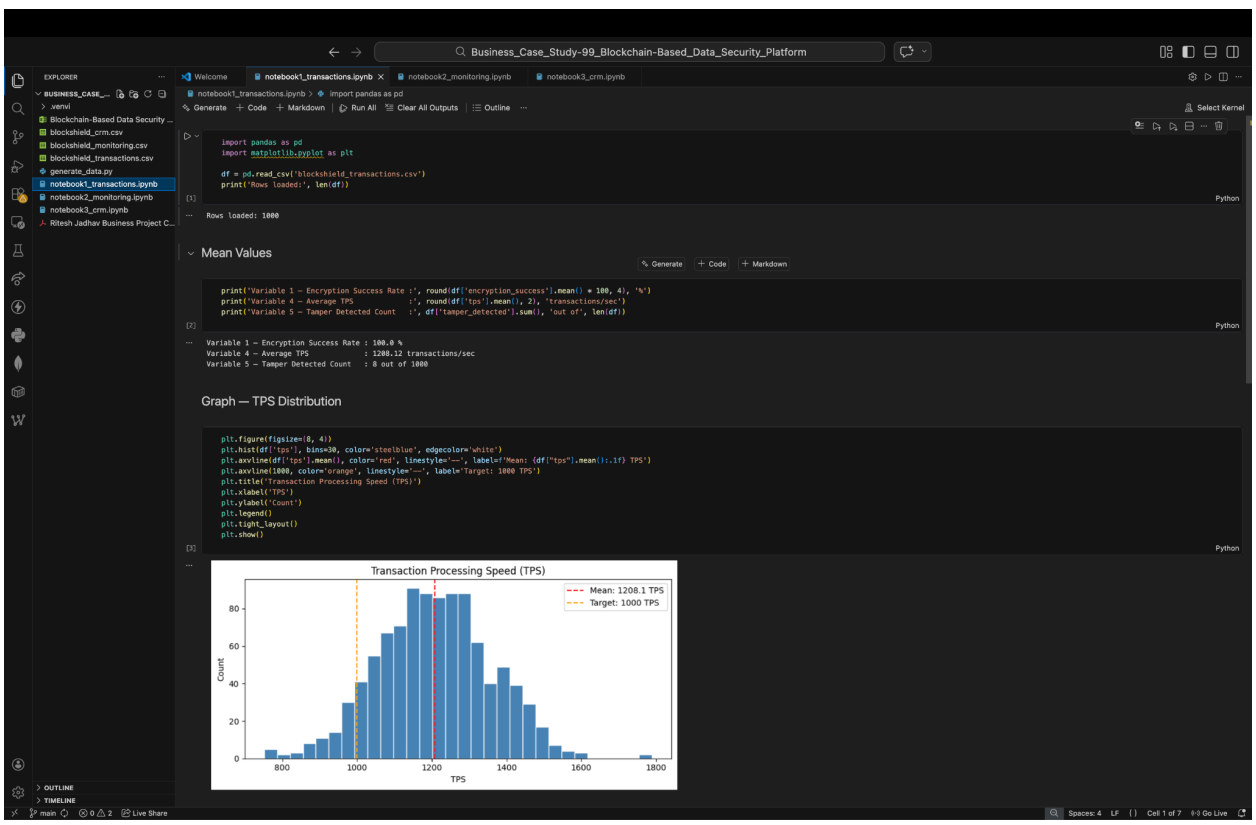
---

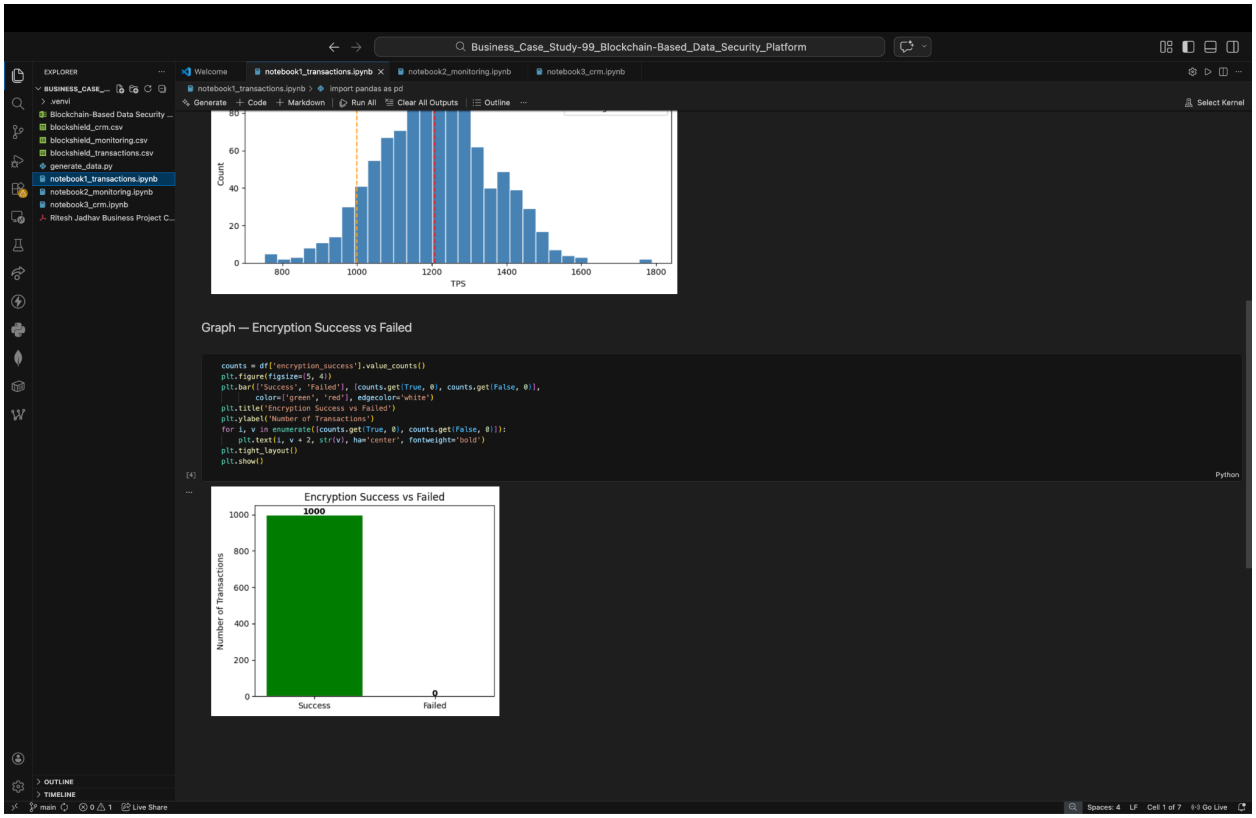
```
print("Generating CRM & Client Satisfaction Dataset")
clients = []

for i in range(1, NUM_CLIENTS + 1):
    industry = random.choice(INDUSTRIES)
    months   = random.randint(1, 12)
    satisfaction = round(
        max(1.0, min(10.0, np.random.normal(loc=8.4, scale=0.8))), 1
    )
    tickets   = max(0, int(np.random.poisson(lam=2)))
    renewal_p = round(max(0.1, min(0.99, 0.5 + (satisfaction - 5) * 0.09)), 2)
    clv       = round(3 * (1 + satisfaction / 5), 2)
    # Churn risk category
    if satisfaction >= 8.0:
        risk = 'LOW'
    elif satisfaction >= 6.5:
        risk = 'MEDIUM'
    else:
        risk = 'HIGH'


    clients.append({
        'client_id'           : f'CLIENT_{i:03d}',
        'industry'            : industry,
        'subscription_months' : months,
        'satisfaction_score'  : satisfaction,
        'support_tickets'     : tickets,
        'renewal_probability' : renewal_p,
        'clv_lakhs'           : clv,
        'churn_risk'          : risk
```

```
    })

df_clients = pd.DataFrame(clients)
print(f"   Done — {len(df_clients):,} client records created.")


df_tx.to_csv('blockshield_transactions.csv', index=False)
df_system.to_csv('blockshield_monitoring.csv', index=False)
df_clients.to_csv('blockshield_crm.csv', index=False)
print("  All 3 CSV files saved successfully!")
```

## d. Jupyter NoteBook For the CSV analysis

### 1. For the CSV of Transaction






2. For the CSV of Monitoring

Business_Case_Study-99_Blockchain-Based_Data_Security_Platform

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('blockshield_monitoring.csv')
print('Rows loaded:', len(df))
```

```
Rows loaded: 365
```

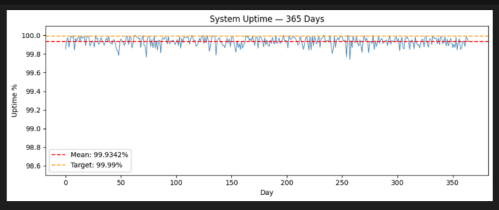
## Mean Values

```python
print('Variable 2 — Average System Uptime        :', round(df['uptime_pct'].mean(), 4), '%')
print('Variable 3 — Avg Unauthorized Attempts/Day:', round(df['unauthorized_attempts'].mean(), 1))
print('Variable 6 — Avg Incident Response Time   :', round(df['incident_response_min'].mean(), 2), 'minutes')
```

```
Variable 2 — Average System Uptime        : 99.9342 %
Variable 3 — Avg Unauthorized Attempts/Day: 22.3
Variable 6 — Avg Incident Response Time   : 3.11 minutes
```

## Graph — System Uptime over 365 Days

```python
plt.figure(figsize=(10, 4))
plt.plot(df['uptime_pct'], color='steelblue', linewidth=0.9)
plt.axhline(df['uptime_pct'].mean(), color='red', linestyle='--',
            label='Mean: {df["uptime_pct"].mean():.4f}%')
plt.axhline(99.99, color='orange', linestyle='--', label='Target: 99.99%')
plt.title('System Uptime — 365 Days')
plt.xlabel('Day')
plt.ylabel('Uptime %')
plt.ylim(98.5, 100.1)
plt.legend()
plt.tight_layout()
plt.show()
```



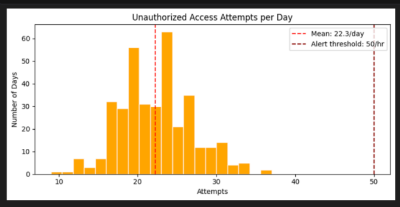

## Graph — Unauthorized Attempts per Day

```python
plt.figure(figsize=(8, 4))
plt.hist(df['unauthorized_attempts'], bins=20, color='orange', edgecolor='white')
plt.axvline(df['unauthorized_attempts'].mean(), color='red', linestyle='--',
            label='Mean: {df["unauthorized_attempts"].mean():.1f}/day')
plt.axvline(50, color='darkred', linestyle='--', label='Alert threshold: 50/hr')
plt.title('Unauthorized Access Attempts per Day')
```

Business_Case_Study-99_Blockchain-Based_Data_Security_Platform

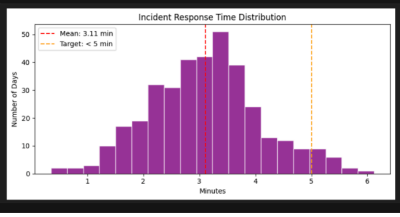
## Graph — Unauthorized Attempts per Day

```python
plt.figure(figsize=(8, 4))
plt.hist(df['unauthorized_attempts'], bins=20, color='orange', edgecolor='white')
plt.axvline(df['unauthorized_attempts'].mean(), color='red', linestyle='--',
            label='Mean: {df["unauthorized_attempts"].mean():.1f}/day')
plt.axvline(50, color='darkred', linestyle='--', label='Alert threshold: 50/hr')
plt.title('Unauthorized Access Attempts per Day')
plt.xlabel('Attempts')
plt.ylabel('Number of Days')
plt.legend()
plt.tight_layout()
plt.show()
```




## Graph — Incident Response Time

```python
plt.figure(figsize=(8, 4))
plt.hist(df['incident_response_min'], bins=20, color='purple', edgecolor='white', alpha=0.8)
plt.axvline(df['incident_response_min'].mean(), color='red', linestyle='--',
            label='Mean: {df["incident_response_min"].mean():.2f} min')
plt.axvline(5, color='orange', linestyle='--', label='Target: < 5 min')
plt.title('Incident Response Time Distribution')
plt.xlabel('Minutes')
plt.ylabel('Number of Days')
plt.legend()
plt.tight_layout()
plt.show()
```



# 3. For the CSV of the CRM

Business_Case_Study-99_Blockchain-Based_Data_Security_Platform

notebook3_crm.ipynb > import pandas as pd

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('blockshield_crm.csv')
print('Rows loaded:', len(df))
```

```
Rows loaded: 200
```

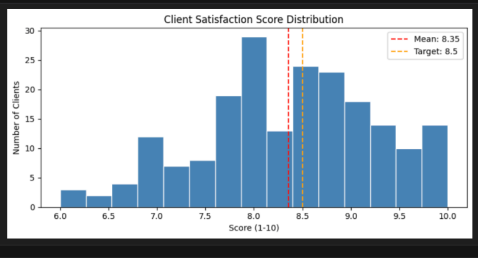
## Mean Values

```python
print('Variable 7 — Avg Client Satisfaction Score:', round(df['satisfaction_score'].mean(), 2), '/ 10')
print('            Avg Renewal Probability       :', round(df['renewal_probability'].mean() * 100, 1), '%')
print('            Avg Customer Lifetime Value   :', round(df['clv_lakhs'].mean(), 2), 'Rs. Lakhs')
```

```
Variable 7 — Avg Client Satisfaction Score: 8.35 / 10
            Avg Renewal Probability       : 80.2 %
            Avg Customer Lifetime Value   : 8.01 Rs. Lakhs
```

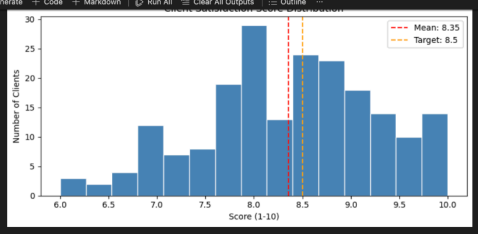
## Graph — Client Satisfaction Score

```python
plt.figure(figsize=(8, 4))
plt.hist(df['satisfaction_score'], bins=15, color='steelblue', edgecolor='white')
plt.axvline(df['satisfaction_score'].mean(), color='red', linestyle='--',
            label=f'Mean: {df["satisfaction_score"].mean():.2f}')
plt.axvline(8.5, color='orange', linestyle='--', label='Target: 8.5')
plt.title('Client Satisfaction Score Distribution')
plt.xlabel('Score (1-10)')
plt.ylabel('Number of Clients')
plt.legend()
plt.tight_layout()
plt.show()
```



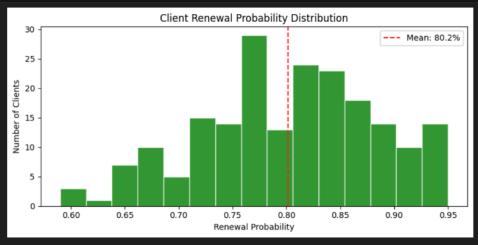

Business_Case_Study-99_Blockchain-Based_Data_Security_Platform

notebook3_crm.ipynb > import pandas as pd




## Graph — Renewal Probability

```python
plt.figure(figsize=(8, 4))
plt.hist(df['renewal_probability'], bins=15, color='green', edgecolor='white', alpha=0.8)
plt.axvline(df['renewal_probability'].mean(), color='red', linestyle='--',
            label=f'Mean: {df["renewal_probability"].mean()*100:.1f}%')
plt.title('Client Renewal Probability Distribution')
plt.xlabel('Renewal Probability')
plt.ylabel('Number of Clients')
plt.legend()
plt.tight_layout()
plt.show()
```



## Section C : CRM and Business Impact

**5. How do strong encryption, reliable system performance, and secure data storage affect client satisfaction, retention, and long-term business growth?**

Ans → This can be explained as below

| |
|---|
| Strong Encryption → Client Trust → High Satisfaction → Renewal → Long-Term Revenue Growth |

1. **Strong Encryption -**
    - **a.** AES-256 encryption ensures government-level data security for banks and hospitals.
    - **b.** 99.95%+ encryption success rate prevents exposure of sensitive records.
    - **c.** Reduces risk of GDPR/RBI compliance violations and fines up to ₹15 Crores.
    - **d.** Helps clients pass ISO 27001, SOC 2, and HIPAA audits.
    - **e.** Certified clients often refer new customers, lowering acquisition cost.
2. **Reliable System Performace -**
    - a. 99.9% uptime SLA (max 8.76 hours downtime/year) expected by enterprise clients.
    - b. AI monitoring detects issues early and sends alerts before clients notice.
    - c. Ensures smooth operations for banks and government organizations.
    - d. 1000+ TPS processing speed supports high-volume transactions.
    - e. Leads to higher retention and premium contract renewals.

3. **Secure Data Storage -**
    a. Immutable blockchain storage provides tamper-proof audit trails.
    b. Helps clients pass HIPAA and RBI inspections.
    c. Prevents data breaches and ensures long-term data integrity.
    d. Builds reputation and reliability for BlockShield.
4. **Business Impact -**
    a. High satisfaction leads to long-term contracts and renewals.
    b. 3+ year clients generate 3–5× higher lifetime value.
    c. Strong security reputation attracts new clients through referrals.
    d. Results in sustainable long-term business growth.

CRM Impact Summary Table

| Sr. No | Technical Driver | CRM Outcome | Retention Impact | Revenue Impact |
|---|---|---|---|---|
| 1 | AES-256 Encryption >= 99.95% | High client trust | Low churn risk | Premium pricing justified |
| 2 | System Uptime >= 99.9% | High satisfaction score | Automatic renewal | Multi-year contracts |
| 3 | Zero tamper events | Regulatory compliance | Deep integration | 3-5x higher CLV |
| 4 | TPS >= 1000 | Smooth operations | No performance churn | Enterprise upsell |
| 5 | AI proactive alerts | White-glove experience | Word-of-mouth referral | Zero CAC growth |

## Based on your analysis, recommend whether the company should launch the blockchain-based data security platform and justify your decision.

### YES — BlockShield Should Launch the Platform

Why ?
→

1. The financial and technical analysis overwhelmingly supports launching the platform. Year 1 records a net loss of Rs.2 Crores due to the initial Rs.6 Crore investment — this is entirely expected and normal for a capital-intensive technology startup. From Year 2 onwards, the platform generates a net annual profit of Rs.4 Crores, with break-even achieved in approximately 1.5 years. This is a strong financial profile.

2. The Indian enterprise cybersecurity market is underserved at the blockchain layer. Banks, hospitals, and government agencies face mounting cyber threats and regulatory pressure — exactly the pain points BlockShield solves. Being a first mover with a certified, compliance-ready platform creates a durable competitive advantage.

3. The SaaS model is highly scalable: each 50 additional clients adds Rs.1.5 Crores in revenue with minimal added cost. As the client base grows and the AI analytics system matures, CLV increases, churn decreases, and referral-driven growth compounds. This is a sustainable, long-term business.

## Key Condition for Success

- Maintain encryption success rate above 99.95% and uptime above 99.9% from Day 1
- Obtain ISO 27001 and SOC 2 certifications within the first 12 months
- Deploy the AI analytics layer by Month 6 to enable proactive churn prevention
- Target 400 clients by Year 3 to achieve Rs.10 Crores+ annual profit